

Guard App — API Integration

Full service layer with Axios instance, all guard-specific service modules, and TypeScript interfaces.

1. Axios Instance

```
// src/api/client.ts
import axios, { AxiosInstance, InternalAxiosRequestConfig,
  AxiosError } from 'axios';
import * as SecureStore from 'expo-secure-store';
import { router } from 'expo-router';
import { API_URL } from '@/constants/config';

const TOKEN_KEY = 'bsecure_guard_token';

export const apiClient: AxiosInstance = axios.create({
  baseURL: API_URL,
  timeout: 15_000,
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json',
  },
});

// — Request interceptor: attach JWT
apiClient.interceptors.request.use(
  async (config: InternalAxiosRequestConfig) => {
    const token = await SecureStore.getItemAsync(TOKEN_KEY);
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// — Response interceptor: handle auth errors
apiClient.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    if (error.response?.status === 401) {

```

```

        // Token expired or invalid – clear and redirect to login
        await SecureStore.deleteItemAsync(TOKEN_KEY);
        router.replace('/(auth)/login');
    }
    return Promise.reject(error);
}
);

// src/constants/config.ts
export const API_URL = process.env.EXPO_PUBLIC_API_URL ??
    'https://api.bsecure.in/api/v1';
export const WS_URL = process.env.EXPO_PUBLIC_WS_URL ?? 'wss://
    api.bsecure.in/ws';
export const GOOGLE_MAPS_KEY =
    process.env.EXPO_PUBLIC_GOOGLE_MAPS_API_KEY ?? '';
export const BACKGROUND_LOCATION_TASK =
    'BSECURE_GUARD_LOCATION_TASK';
export const MIN_PAYOUT_AMOUNT = 500;
export const BOOKING_REQUEST_TIMEOUT_MS = 30_000;

```

2. TypeScript Interfaces

```

// src/types/guard.ts

export type VerificationStatus =
    | 'unverified'
    | 'pending'
    | 'under_review'
    | 'approved'
    | 'rejected';

export type GuardTier = 'basic' | 'standard' | 'premium' |
    'elite';

export interface Guard {
    id: string;
    full_name: string;
    phone: string;
    email?: string;
    photo_url?: string;
    date_of_birth?: string;
    address?: string;
    city?: string;
    skills: string[];
    experience_years: number;
    verification_status: VerificationStatus;
    tier: GuardTier;
    rating: number;
    total_bookings: number;
    total_earnings: number;
    is_online: boolean;
}

```

```
    availability_schedule: AvailabilitySchedule;
    created_at: string;
}
```

```
export interface AvailabilitySchedule {
    monday: DaySchedule;
    tuesday: DaySchedule;
    wednesday: DaySchedule;
    thursday: DaySchedule;
    friday: DaySchedule;
    saturday: DaySchedule;
    sunday: DaySchedule;
}
```

```
export interface DaySchedule {
    enabled: boolean;
    start_time: string; // "HH:MM"
    end_time: string;   // "HH:MM"
}
```

```
export interface PersonalInfoPayload {
    full_name: string;
    date_of_birth: string; // "YYYY-MM-DD"
    address: string;
    city: string;
    skills: string[];
    experience_years: number;
}
```

```
export interface VerificationStatusResponse {
    status: VerificationStatus;
    rejection_reasons?: string[];
    reviewed_at?: string;
}
```

// — Booking Types

```
export type BookingStatus =
    | 'pending'
    | 'accepted'
    | 'en_route'
    | 'arrived'
    | 'started'
    | 'completed'
    | 'cancelled';
```

```
export interface IncomingRequest {
    id: string;
    user_id: string;
    user_name: string;
    user_photo: string | null;
    user_rating: number;
}
```

```

    booking_type: string;
    pickup_location: { lat: number; lon: number };
    pickup_address: string;
    distance_km: number;
    duration_hours: number;
    estimated_earnings: number;
    notes?: string;
    expires_at: string;
}

export interface ActiveBooking {
    id: string;
    user_id: string;
    user_name: string;
    user_photo: string | null;
    user_phone: string;
    booking_type: string;
    pickup_location: { lat: number; lon: number };
    pickup_address: string;
    status: BookingStatus;
    amount: number;
    accepted_at: string;
    arrived_at?: string;
    started_at?: string;
    completed_at?: string;
}

export interface CompletedBooking extends ActiveBooking {
    duration_minutes: number;
    guard_earnings: number;
    platform_fee: number;
    user_rating?: number;
    user_review?: string;
}

export type FilterPeriod = 'today' | 'week' | 'month';

// — Earnings Types

export interface EarningsSummary {
    today: number;
    this_week: number;
    this_month: number;
    total: number;
    available_balance: number;
    pending_payout: number;
    weekly_chart: WeeklyChartPoint[];
}

export interface WeeklyChartPoint {
    day: string; // "Mon", "Tue", etc.
    amount: number;
}

```

```
}
```

```
export interface PayoutRequest {  
  id: string;  
  amount: number;  
  status: 'pending' | 'processing' | 'completed' | 'failed';  
  bank_account_last4?: string;  
  created_at: string;  
  processed_at?: string;  
}
```

```
// — Document Types
```

```
export type DocumentType =  
  | 'aadhaar_front'  
  | 'aadhaar_back'  
  | 'pan_card'  
  | 'police_verification'  
  | 'profile_photo';
```

```
export type DocumentStatus = 'pending' | 'uploading' | 'uploaded'  
  | 'verified' | 'rejected';
```

```
export interface GuardDocument {  
  id: string;  
  document_type: DocumentType;  
  status: DocumentStatus;  
  s3_key?: string;  
  url?: string;  
  rejection_reason?: string;  
  uploaded_at?: string;  
  verified_at?: string;  
}
```

```
export interface UploadURLRequest {  
  document_type: string;  
  file_name: string;  
  content_type: string;  
}
```

```
export interface UploadURLResponse {  
  upload_url: string; // Pre-signed S3 PUT URL  
  s3_key: string;  
}
```

```
export interface ConfirmUploadRequest {  
  document_type: string;  
  s3_key: string;  
}
```

```
// — Location Types
```

```
export interface LocationUpdate {
  booking_id: string;
  lat: number;
  lon: number;
  heading: number | null;
  speed: number | null;
  accuracy: number | null;
  timestamp: string;
}
```

3. Guard Profile Service

```
// src/api/guardProfileService.ts
import { apiClient } from './client';
import type { Guard, PersonalInfoPayload, AvailabilitySchedule }
  from '@types/guard';

export const guardProfileService = {
  /**
   * Fetch the current guard's full profile.
   */
  getMyProfile: async (): Promise<Guard> => {
    const { data } = await apiClient.get<Guard>('/guards/me/');
    return data;
  },

  /**
   * Update guard profile (name, city, skills, experience, etc.)
   */
  updateProfile: async (payload: Partial<PersonalInfoPayload>):
    Promise<Guard> => {
    const { data } = await apiClient.patch<Guard>('/guards/me/',
      payload);
    return data;
  },

  /**
   * Toggle online/offline status.
   */
  setOnlineStatus: async (isOnline: boolean):
    Promise<{ is_online: boolean }> => {
    const { data } = await apiClient.post<{ is_online: boolean }>(<
      '/guards/me/status/',
      { is_online: isOnline }
    );
    return data;
  },

  /**
   * Get the guard's weekly availability schedule.
   */
}
```

```

    */
    getAvailabilitySchedule: async ():
        Promise<AvailabilitySchedule> => {
            const { data } = await apiClient.get<AvailabilitySchedule>(
                '/guards/me/availability/'
            );
            return data;
        },

    /**
     * Update the guard's weekly availability schedule.
     */
    updateAvailability: async (
        schedule: AvailabilitySchedule
    ): Promise<AvailabilitySchedule> => {
        const { data } = await apiClient.put<AvailabilitySchedule>(
            '/guards/me/availability/',
            schedule
        );
        return data;
    },
};

```

4. Booking Service

```

// src/api/bookingService.ts
import { apiClient } from './client';
import type {
    ActiveBooking,
    CompletedBooking,
    FilterPeriod,
} from '@types/guard';

export const bookingService = {
    /**
     * Accept an incoming booking request.
     * Returns 409 Conflict if another guard already accepted.
     */
    acceptBooking: async (bookingId: string):
        Promise<ActiveBooking> => {
        const { data } = await apiClient.post<ActiveBooking>(
            `/bookings/${bookingId}/accept/`
        );
        return data;
    },

    /**
     * Decline an incoming booking request.
     */
    declineBooking: async (

```

```

    bookingId: string,
    reason?: string
  ): Promise<void> => {
    await apiClient.post(`/bookings/${bookingId}/decline/`, {
      reason: reason ?? 'guard_declined',
    });
  },

  /**
   * Mark that the guard has arrived at the pickup location.
   */
  markArrived: async (bookingId: string): Promise<ActiveBooking>
    => {
    const { data } = await apiClient.post<ActiveBooking>(
      `/bookings/${bookingId}/arrived/`
    );
    return data;
  },

  /**
   * Start the service (user has been met, service begins).
   */
  startBooking: async (bookingId: string): Promise<ActiveBooking>
    => {
    const { data } = await apiClient.post<ActiveBooking>(
      `/bookings/${bookingId}/start/`
    );
    return data;
  },

  /**
   * Mark the service as completed.
   */
  completeBooking: async (bookingId: string):
    Promise<CompletedBooking> => {
    const { data } = await apiClient.post<CompletedBooking>(
      `/bookings/${bookingId}/complete/`
    );
    return data;
  },

  /**
   * Fetch the guard's current active booking (for restoration
   after restart).
   */
  getActiveBooking: async (): Promise<ActiveBooking | null> => {
    try {
      const { data } = await apiClient.get<ActiveBooking>(`/
        bookings/active/`);
      return data;
    } catch (err: any) {
      if (err.response?.status === 404) return null;
      throw err;
    }
  }
}

```

```

    }
  },

  /**
   * Get a specific completed booking's detail.
   */
  getBookingDetail: async (bookingId: string):
    Promise<CompletedBooking> => {
    const { data } = await apiClient.get<CompletedBooking>(
      `/bookings/${bookingId}/`
    );
    return data;
  },

  /**
   * Get booking history filtered by time period.
   */
  getHistory: async (
    period: FilterPeriod = 'week'
  ): Promise<CompletedBooking[]> => {
    const { data } = await apiClient.get<CompletedBooking[]>(
      '/bookings/history/',
      { params: { period } }
    );
    return data;
  },
};

```

5. Earnings Service

```

// src/api/earningsService.ts
import { apiClient } from './client';
import type { EarningsSummary, PayoutRequest } from '@types/guard';

export const earningsService = {
  /**
   * Fetch earnings summary: today, week, month, total, available
   * balance.
   */
  getSummary: async (): Promise<EarningsSummary> => {
    const { data } = await apiClient.get<EarningsSummary>(
      '/guards/me/earnings/summary/'
    );
    return data;
  },

  /**
   * Fetch payout history.
   */
  getPayoutHistory: async (): Promise<PayoutRequest[]> => {

```

```

    const { data } = await apiClient.get<PayoutRequest[]>(
      '/guards/me/payouts/'
    );
    return data;
  },

  /**
   * Request a payout of available balance.
   * Returns 400 if balance is below the minimum threshold.
   */
  requestPayout: async (): Promise<PayoutRequest> => {
    const { data } = await apiClient.post<PayoutRequest>(
      '/guards/me/payouts/request/'
    );
    return data;
  },
};

```

6. Document Service

```

// src/api/documentService.ts
import { apiClient } from './client';
import type {
  GuardDocument,
  UploadURLRequest,
  UploadURLResponse,
  ConfirmUploadRequest,
} from '@types/guard';

export const documentService = {
  /**
   * List all documents submitted by the guard with their
   * verification status.
   */
  getDocuments: async (): Promise<GuardDocument[]> => {
    const { data } = await apiClient.get<GuardDocument[]>(
      '/guards/me/documents/'
    );
    return data;
  },

  /**
   * Request a pre-signed S3 URL to upload a document directly.
   * The guard app uploads the file directly to S3, never through
   * our server.
   */
  getUploadURL: async (
    payload: UploadURLRequest
  ): Promise<UploadURLResponse> => {
    const { data } = await apiClient.post<UploadURLResponse>(

```

```

        '/guards/me/documents/upload-url/',
        payload
    );
    return data;
},

/**
 * After successfully uploading to S3, confirm the upload so the
 * backend
 * can store the s3_key and trigger verification.
 */
confirmUpload: async (payload: ConfirmUploadRequest):
    Promise<GuardDocument> => {
    const { data } = await apiClient.post<GuardDocument>(
        '/guards/me/documents/confirm/',
        payload
    );
    return data;
},

/**
 * Re-upload a rejected document (same flow as initial upload).
 */
reuploadDocument: async (
    documentId: string,
    payload: ConfirmUploadRequest
): Promise<GuardDocument> => {
    const { data } = await apiClient.patch<GuardDocument>(
        `/guards/me/documents/${documentId}/`,
        payload
    );
    return data;
},
};

```

7. Location Service

Used by the background task when sending updates via HTTP (not WebSocket).

```

// src/api/locationService.ts
import { API_URL } from '@constants/config';
import type { LocationUpdate } from '@types/guard';

/**
 * Send a location update from the background task.
 *
 * IMPORTANT: This function intentionally uses `fetch` instead of
 * Axios.
 *
 * Axios may not be available in all background task JS contexts.

```

```

* fetch is guaranteed to be available as a global.
*/
export async function sendLocationUpdate(
  update: LocationUpdate,
  token: string
): Promise<void> {
  const response = await fetch(`${API_URL}/tracking/location/`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify(update),
    signal: AbortSignal.timeout(8_000), // 8s max
  });

  if (!response.ok) {
    throw new Error(`Location update failed: HTTP $
      {response.status}`);
  }
}

```

8. Onboarding Service

```

// src/api/onboardingService.ts
import { apiClient } from './client';
import type {
  Guard,
  PersonalInfoPayload,
  VerificationStatusResponse,
} from '@types/guard';

export const onboardingService = {
  /**
   * Submit Step 1: personal information and skills.
   */
  submitPersonalInfo: async (payload: PersonalInfoPayload):
    Promise<Guard> => {
    const { data } = await apiClient.post<Guard>(
      '/onboarding/personal-info/',
      payload
    );
    return data;
  },

  /**
   * Mark onboarding as submitted (called after all documents are
     uploaded).
   * Transitions verification_status from 'unverified' to
     'pending'.
   */
}

```

```

submitForReview: async (): Promise<{ message: string }> => {
  const { data } = await apiClient.post<{ message: string }>(
    '/onboarding/submit/'
  );
  return data;
},

/**
 * Poll verification status during the "under review" screen.
 */
getVerificationStatus: async ():
  Promise<VerificationStatusResponse> => {
  const { data } = await
    apiClient.get<VerificationStatusResponse>(
      '/guards/me/verification-status/'
    );
  return data;
},
};

```

9. Auth Service

```

// src/api/authService.ts
import { apiClient } from './client';
import type { Guard } from '@types/guard';

interface OTPRequestResponse {
  message: string;
  expires_in: number; // seconds
}

interface OTPVerifyResponse {
  token: string;
  guard: Guard;
}

export const authService = {
  /**
   * Request OTP for a guard phone number.
   * Only registered guard numbers receive an OTP.
   */
  requestGuardOtp: async (phone: string):
    Promise<OTPRequestResponse> => {
    const { data } = await apiClient.post<OTPRequestResponse>(
      '/auth/guard/otp/request/',
      { phone }
    );
    return data;
  },

  /**

```

```

    * Verify OTP and receive JWT + guard profile.
    */
    verifyGuardOtp: async (
      phone: string,
      otp: string
    ): Promise<OTPVerifyResponse> => {
      const { data } = await apiClient.post<OTPVerifyResponse>(
        '/auth/guard/otp/verify/',
        { phone, otp }
      );
      return data;
    },

    /**
     * Logout: revoke token on the backend.
     */
    logout: async (): Promise<void> => {
      await apiClient.post('/auth/guard/logout/');
    },
  };

```

10. React Query Key Conventions

Keep query keys centralized to avoid typos and enable precise invalidation:

```

// src/constants/queryKeys.ts
export const QUERY_KEYS = {
  guardProfile: ['guard-profile'] as const,
  activeBooking: ['active-booking'] as const,
  bookingDetail: (id: string) => ['booking', id] as const,
  bookingHistory: (period: string) => ['booking-history', period]
    as const,
  earningsSummary: ['earnings-summary'] as const,
  payoutHistory: ['payout-history'] as const,
  documents: ['guard-documents'] as const,
  verificationStatus: ['verification-status'] as const,
  availabilitySchedule: ['availability-schedule'] as const,
};

```

11. Error Handling Patterns

```

// src/lib/apiErrors.ts
import { AxiosError } from 'axios';

export interface APIError {
  message: string;
  code?: string;
  field_errors?: Record<string, string[]>;
}

```

```

    status: number;
}

export function parseAPIError(error: unknown): APIError {
  if (error instanceof AxiosError && error.response) {
    const status = error.response.status;
    const data = error.response.data;

    return {
      status,
      message: data?.detail ?? data?.message ??
        getDefaultMessage(status),
      code: data?.code,
      field_errors: data?.errors ?? data?.field_errors,
    };
  }

  return {
    status: 0,
    message: 'Network error. Please check your connection.',
  };
}

function getDefaultMessage(status: number): string {
  const messages: Record<number, string> = {
    400: 'Invalid request. Please check your input.',
    401: 'Your session has expired. Please log in again.',
    403: 'You do not have permission to perform this action.',
    404: 'The requested resource was not found.',
    409: 'This action conflicts with the current state.',
    422: 'Validation failed. Please check your input.',
    429: 'Too many requests. Please wait a moment.',
    500: 'Server error. Please try again later.',
  };
  return messages[status] ?? 'An unexpected error occurred.';
}

```

12. API Endpoint Reference

Method	Endpoint	Service	Description
POST	/auth/guard/ otp/request/	authService	Request OTP
POST	/auth/guard/ otp/verify/	authService	Verify OTP, get token
POST	/auth/guard/ logout/	authService	Invalidate token
GET	/guards/me/	guardProfileService	

Method	Endpoint	Service	Description
			Get guard profile
PATCH	/guards/me/	guardProfileService	Update profile
POST	/guards/me/status/	guardProfileService	Toggle online/offline
GET	/guards/me/availability/	guardProfileService	Get weekly schedule
PUT	/guards/me/availability/	guardProfileService	Update weekly schedule
POST	/onboarding/personal-info/	onboardingService	Submit personal info
POST	/onboarding/submit/	onboardingService	Submit for review
GET	/guards/me/verification-status/	onboardingService	Poll verification status
GET	/guards/me/documents/	documentService	List documents
POST	/guards/me/documents/upload-url/	documentService	Get S3 pre-signed URL
POST	/guards/me/documents/confirm/	documentService	Confirm S3 upload
POST	/bookings/{id}/accept/	bookingService	Accept booking
POST	/bookings/{id}/decline/	bookingService	Decline booking
POST	/bookings/{id}/arrived/	bookingService	Mark arrived
POST	/bookings/{id}/start/	bookingService	Start service
POST	/bookings/{id}/complete/	bookingService	Complete service
GET	/bookings/active/	bookingService	Get active booking
GET	/bookings/{id}/	bookingService	Get booking detail
GET	/bookings/history/	bookingService	Get history (filtered)

Method	Endpoint	Service	Description
GET	/guards/me/earnings/summary/	earningsService	Get earnings summary
GET	/guards/me/payouts/	earningsService	Get payout history
POST	/guards/me/payouts/request/	earningsService	Request payout
POST	/tracking/location/	locationService	Send location update